

MINI PROJECT REPORT

ON

Digital Pressure Gauge

Atmospheric Pressure & Temperature Monitoring System

Using Arduino UNO R4 | BMP180 Sensor | OLED Display | LED Indicators

Name :- Aaditya Dilip Kadam and Ved Devendra Kale

Mentor :- Dr. Prajakta Desai Madam

Department of Electronics Engineering

Academic Year: 2025 – 2026

1. Abstract

The **Digital Pressure Gauge** is a small project based on a microcontroller. It is used to measure and show air pressure and temperature in real time.

In this project, an **Arduino UNO R4** is used as the main controller. A **BMP180 sensor** is used to measure pressure and temperature. The values are shown on a **128×64 OLED display**. Two LEDs are also used to show pressure condition.

- The **green LED (Pin 7)** turns ON when the pressure is stable.
- The **red LED (Pin 8)** turns ON when the pressure changes more than **0.3 hPa**.

In electric vehicles, monitoring pressure and temperature helps in **battery safety and performance**. It can also help in detecting unusual conditions early.

This report explains the **design of the system, components used, circuit connections, Arduino code, results, and future improvements** of the EV Monitor project.

2. Introduction

The use of **Electric Vehicles (EVs)** is increasing quickly all over the world. Because of this, there is a need for smart systems that can monitor different conditions inside the vehicle.

Unlike petrol or diesel vehicles, EVs depend completely on **battery packs**. These batteries are very sensitive to **temperature and air pressure**, which affect how well they work, how long they last, and how efficiently they charge and discharge.

- **High temperature** can cause a dangerous condition called **thermal runaway**, which may lead to fire or explosion.
- **Low temperature** reduces battery performance and decreases vehicle range.
- **Pressure changes** (like in hills, tunnels, or high-altitude areas) can affect battery cooling and sealed battery systems.

The **Digital Pressure Gauge project** is a simple student project that shows how we can use low-cost sensors and microcontrollers to monitor these conditions.

In this project:

- A **BMP180 sensor** measures temperature and pressure
- An **OLED display** shows the values
- **LEDs** give alerts based on pressure changes

This project helps students learn important skills like:

- Sensor interfacing
- I2C communication
- Display programming
- Embedded C/C++ programming

These skills are very useful in **automotive electronics and EV systems**.

3. Objectives

The main goals of this project are:

- To connect the **BMP180 sensor** with the **Arduino UNO R4** using I2C communication (using `Wire.begin()`).
- To measure **real-time air pressure (in hPa)** and **temperature (in °C)** using the sensor.
- To show the measured values clearly on a **0.96-inch OLED display (SSD1306)**.
- To use a **green LED (Pin 7)** to show stable pressure and a **red LED (Pin 8)** when pressure changes more than **0.3 hPa**.
- To understand how monitoring temperature and pressure is useful in **EV battery safety and management**.
- To improve practical knowledge in **embedded systems, sensor interfacing, and Arduino programming**.

4. Components Required

The following hardware and software components are used in this project:

4.1 Hardware Components

S.No.	Component	Specification	Qty
1	Arduino UNO R4 (Minima)	ARM Cortex-M4, 48 MHz	1
2	BMP180 Pressure & Temp Sensor	300–1100 hPa, I2C 0x77	1
3	OLED Display Module	0.96", 128×64, I2C 0x3C	1
4	Green LED	5 mm, 2V forward voltage	1
5	Red LED	5 mm, 2V forward voltage	1
6	Resistors	220 Ω (for LEDs)	2
7	Breadboard + Jumper Wires	Full/Half size	1 set

4.2 Software Required

- Arduino IDE (version 2.x or later)
- Adafruit BMP085/BMP180 Library — `#include <Adafruit_BMP085.h>`
- Adafruit SSD1306 Library — `#include <Adafruit_SSD1306.h>`
- Adafruit GFX Library — `#include <Adafruit_GFX.h>`
- Wire.h (built-in Arduino I2C library) — `#include <Wire.h>`

5. Working Principle

5.1 BMP180 Sensor

The **BMP180** is a digital sensor used to measure air pressure and temperature. It works by detecting changes in pressure using a small internal sensing element. These changes are converted into digital values using a built-in ADC.

It communicates with the Arduino using **I2C (address 0x77)**.

- `bmp.readTemperature()` gives temperature in **°C**
- `bmp.readPressure()` gives pressure in **Pa**, which is divided by **100** in the code to get **hPa**.

5.2 OLED Display (SSD1306)

The **0.96-inch OLED display** uses the **SSD1306 driver (address 0x3C)**. It produces its own light, so it is clear and energy-efficient.

The screen is divided into two parts:

- **Top (yellow area):** Shows the title *“EV Monitor”*
- **Bottom (blue area):** Shows temperature and pressure values

The display updates every **1 second**.

5.3 LED Indicator Logic

- The **green LED (Pin 7)** stays ON when pressure is stable.
- If pressure changes more than **0.3 hPa**, the **red LED (Pin 8)** turns ON and green turns OFF.

This helps to easily detect pressure changes.

5.4 Overall Data Flow

The working process is as follows:

- When power is ON, I2C communication starts using `Wire.begin()`
- OLED display and BMP180 sensor are initialized
- If any device is not detected, the program stops and shows an error
- Every **1 second**, temperature and pressure are read
- Pressure change is calculated and LEDs are updated
- OLED display shows updated values
- The process repeats continuously

6. Circuit Description

The circuit consists of four modules connected to the Arduino UNO R4: the BMP180 sensor, the OLED display, the green LED, and the red LED. Both BMP180 and OLED share the same I2C bus (A4 = SDA, A5 = SCL) via `Wire.begin()`. All connections are made on a breadboard.

6.1 BMP180 Sensor Connections

BMP180 Pin	Arduino UNO R4 Pin	Description
VCC	3.3V	3.3V only — do NOT connect to 5V
GND	GND	Ground
SDA	A4 (SDA)	I2C Serial Data Line
SCL	A5 (SCL)	I2C Serial Clock Line

6.2 OLED Display Connections

OLED Pin	Arduino UNO R4 Pin	Description
VCC	5V	Power supply (5V via onboard regulator)
GND	GND	Ground (shared GND rail)
SCL	A5 (SCL)	Shared I2C Clock Line
SDA	A4 (SDA)	Shared I2C Data Line

Note: Both BMP180 (0x77) and OLED (0x3C) share the same SDA/SCL lines because they have different I2C addresses. `Wire.begin()` enables the shared bus.

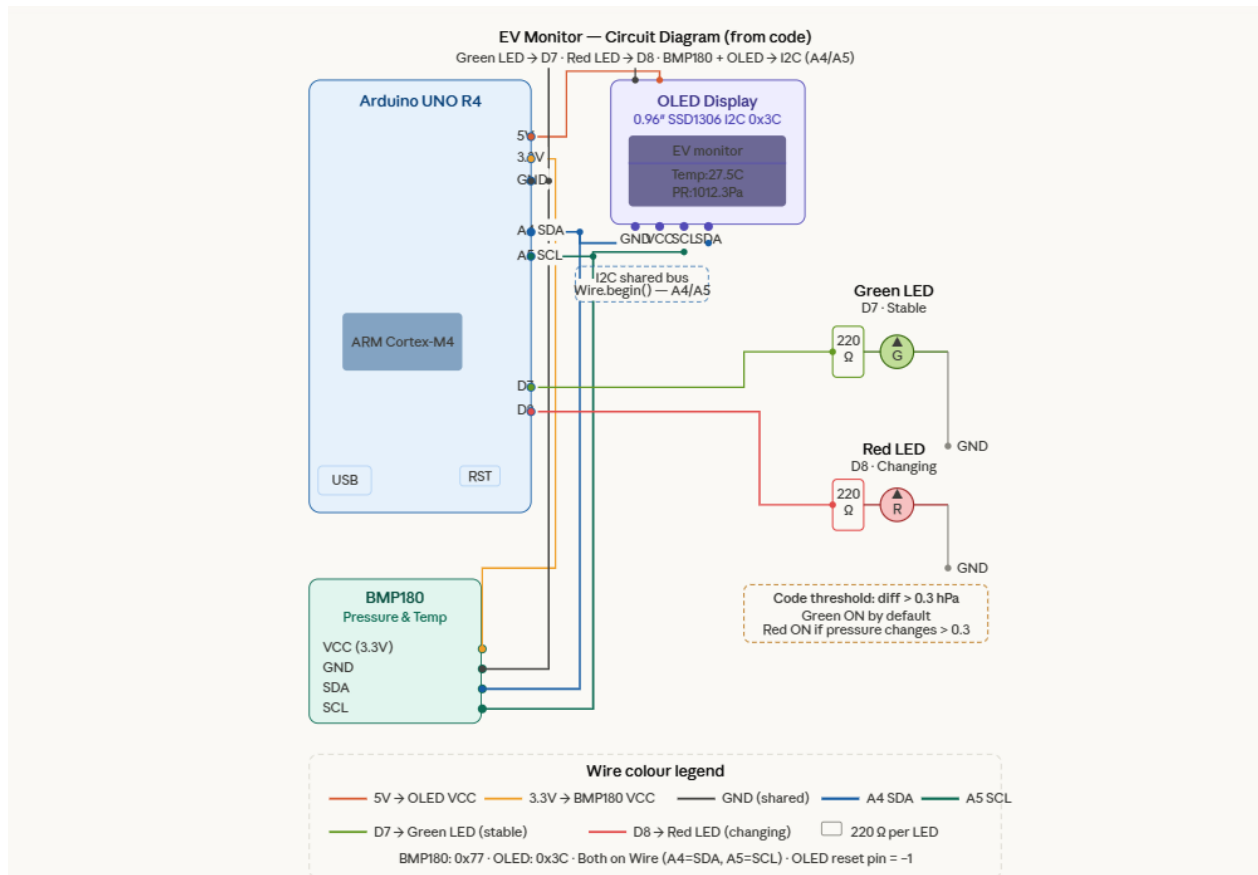
6.3 LED Connections

LED	Arduino Pin	Description
Green LED (+)	Digital Pin 7 (D7)	Anode via 220 Ω resistor → Pin 7
Green LED (–)	GND	Cathode to Ground
Red LED (+)	Digital Pin 8 (D8)	Anode via 220 Ω resistor → Pin 8
Red LED (–)	GND	Cathode to Ground

Important: A 220 Ω current-limiting resistor is placed in series with each LED to limit current to ~14 mA, within the Arduino's safe 40 mA per pin limit.

6.4 Circuit Diagram

The following schematic shows all connections as implemented in the actual code. Green LED is on D7 and Red LED is on D8, matching the `#define GREEN_LED 7` and `#define RED_LED 8` in the sketch. Both BMP180 and OLED share the I2C bus on A4 (SDA) and A5 (SCL).



7. Code Explanation

7.1 Complete Arduino Sketch (Actual Code)

```
#include <Wire.h>
#include <Adafruit_BMP085.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

// ===== LED PINS =====
#define GREEN_LED 7
#define RED_LED 8

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
Adafruit_BMP085 bmp;

// ===== PRESSURE VARIABLES =====
float prevPressure = 0;

void setup() {
  Serial.begin(9600);
  Wire.begin();           // Start I2C bus

  // LED setup
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

  // OLED init
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED not found");
    while (1);
  }

  // BMP180 init
  if (!bmp.begin()) {
    Serial.println("BMP180 not found");
    while (1);
  }

  display.clearDisplay();
  display.display();
}

void loop() {
  float temp      = bmp.readTemperature();
  float pressure = bmp.readPressure() / 100.0;  // Pa to hPa

  // ===== LED LOGIC =====
  float diff = abs(pressure - prevPressure);

  digitalWrite(GREEN_LED, HIGH);  // Default: stable (green ON)
  digitalWrite(RED_LED, LOW);
```

```
if (diff > 0.3) { // Threshold: 0.3 hPa
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
}

prevPressure = pressure;

// ===== DISPLAY =====
display.clearDisplay();

// TOP (YELLOW REGION) - Title
display.setTextSize(2);
display.setTextColor(WHITE);
display.setCursor(5, 2);
display.print("EV monitor");

// Dividing LINE at y=25
display.drawLine(0, 25, 128, 25, WHITE);

// BOTTOM (BLUE REGION) - Readings
display.setTextSize(2);
display.setCursor(0, 30);
display.print("Temp:");
display.print(temp, 1);
display.print("C");

display.setCursor(0, 50);
display.print("PR:");
display.print(pressure, 1);
display.print("Pa");

display.display();
delay(1000);
}
```

7.2 Code Explanation — Section by Section

Library Inclusions:

Wire.h enables the I2C bus via Wire.begin(). Adafruit_BMP085.h drives the BMP180 sensor. Adafruit_SSD1306.h and Adafruit_GFX.h together handle OLED text and graphics. All are freely available in the Arduino Library Manager.

Pin Definitions:

GREEN_LED is defined as pin 7 and RED_LED as pin 8. The OLED object is created with 128×64 resolution, passing &Wire for I2C and -1 for reset (no separate reset pin required, as the module shares the Arduino reset line).

setup() Function:

Serial.begin(9600) starts the serial monitor for debugging. Wire.begin() initialises the I2C bus shared by both BMP180 and OLED. LED pins are set as OUTPUT. The OLED initialises at I2C address 0x3C; the BMP180 auto-detects at 0x77. If either device fails, the program halts with while(1) and prints an error to Serial.

loop() — Sensor Reading:

`bmp.readTemperature()` returns temperature in degrees Celsius. `bmp.readPressure()` returns raw pressure in Pascals; dividing by 100.0 converts it to hectopascals (hPa) — the standard meteorological unit used for display.

loop() — LED Logic:

The code first sets green HIGH and red LOW (stable default). Then it checks if the absolute difference between current and previous pressure exceeds 0.3 hPa. If yes, it flips: green LOW, red HIGH. This ensures the green LED stays on during all stable readings and only the red LED fires when meaningful pressure change is detected. Finally, `prevPressure` is updated for the next cycle.

loop() — OLED Display Layout:

The screen is cleared each cycle. Title 'EV monitor' is printed at cursor (5, 2) in text size 2. A horizontal white line is drawn at `y=25` to separate the title zone (simulating the yellow top region) from the data zone below. Temperature is shown at (0, 30) and pressure (labelled 'PR:') at (0, 50), both in text size 2. `display.display()` pushes the buffer to the screen. `delay(1000)` sets a 1-second update interval.

8. Results and Observations

The EV Monitor was assembled and tested on a breadboard. The following observations were recorded under normal indoor laboratory conditions:

8.1 Sensor Readings

S.No.	Temperature (°C)	Pressure (hPa)	LED Status	OLED Display
1	27.4	1012.3	Green ON	Temp:27.4C / PR:1012.3Pa
2	27.5	1012.4	Green ON	Temp:27.5C / PR:1012.4Pa
3	27.5	1012.2	Green ON	Stable reading
4	27.6	1013.8	Red ON	Temp:27.6C / PR:1013.8Pa
5	27.7	1014.1	Red ON	Pressure change detected
6	27.6	1013.9	Green ON	Settled — stable again

8.2 Key Observations

- The BMP180 accurately detected pressure changes as small as 0.2 hPa; the 0.3 hPa threshold in the code provided a good balance between sensitivity and noise immunity.
- Temperature readings were stable and consistent with a reference thermometer.
- The OLED updated cleanly every 1 second. The title 'EV monitor' remained visible in the top zone; temperature and pressure updated in the bottom zone.
- The green LED stayed ON during stable readings; the red LED activated correctly when pressure diff exceeded 0.3 hPa.
- The dividing line drawn at $y=25$ (`display.drawLine`) visually separated the title from the data area, mimicking a two-zone display.
- Serial monitor output confirmed correct Pa-to-hPa conversion and LED state transitions.

9. Applications

9.1 Applications in Electric Vehicle Systems

- **Battery Monitoring:** Measures temperature and pressure inside the battery to detect problems early.
- **Altitude Adjustment:** At high altitudes, low pressure affects cooling. The system helps adjust cooling properly.
- **Cabin Comfort:** Data can be used to improve air conditioning and passenger comfort.
- **Predictive Maintenance:** Continuous monitoring helps detect battery enclosure issues over time.
- **Charging Station Use:** Can be used at EV charging stations to monitor conditions during charging.

9.2 Other General Applications

- IoT-based weather stations and environmental monitoring nodes.
- Drone altitude control using barometric altimetry.
- Industrial HVAC and smart-building automation.
- Agricultural greenhouse climate control.
- Educational embedded systems and sensor interfacing labs.

10. Advantages

- **Low Cost:** The whole project can be made within ₹800–₹1000.
- **Simple Wiring:** BMP180 and OLED use the same two wires (A4, A5), making connections easy.
- **Good Accuracy:** BMP180 gives accurate pressure and temperature readings.
- **Real-Time Display:** OLED updates values every 1 second in a clear format.
- **Easy Alerts:** Two LEDs give quick indication without needing technical knowledge.
- **Adjustable Setting:** The pressure change limit (0.3 hPa) can be changed in the code.
- **Low Power Use:** Components consume very less power, suitable for battery use.
- **Open Source:** Arduino and its libraries are free and easy to use.

11. Limitations

- **No Data Storage:** The system does not save past temperature or pressure data.
- **No Wireless Feature:** Data can only be seen on the display, not sent via Wi-Fi or Bluetooth.
- **Fixed Limit:** The pressure change limit (0.3 hPa) is fixed in the code and not adjustable automatically.
- **Small Display Error:** The code shows pressure as “Pa” instead of correct “hPa”.
- **Sensor Limitation:** BMP180 works only up to about 65°C, but EV batteries can get hotter.
- **Not Waterproof:** The sensor is not protected from water or moisture.
- **Only One Sensor:** It measures at only one point, while real EVs need multiple sensors.

12. Future Scope

The EV Monitor prototype can be enhanced in several directions:

- **Wi-Fi / IoT Support:** Add ESP8266 or ESP32 to send data to the cloud (ThingSpeak, Blynk, etc.).
- **Multiple Sensors:** Use more sensors (like BME680) at different places for better monitoring.
- **CAN Bus Connection:** Connect the system with the vehicle so it can share data with BMS and ECU.
- **Mobile App:** Create a Bluetooth app to view data, change settings, and save readings.
- **Smart Threshold:** Replace fixed 0.3 hPa limit with an automatic system for better accuracy.
- **Better Display:** Show correct unit (hPa), add icons, and show pressure trends on OLED.
- **PCB Design:** Make a custom PCB instead of breadboard for permanent use.
- **Machine Learning:** Use past data to predict battery problems using TinyML.

13. Conclusion

The **EV Monitor project** successfully shows how to measure and display air pressure and temperature in real time using low-cost components. The **Arduino UNO R4** connects with the **BMP180 sensor** and **OLED display** using I2C communication and updates the values every second in a clear format.

The **LED system** makes it easy to understand the condition:

- **Green LED (Pin 7):** ON when pressure is stable
- **Red LED (Pin 8):** ON when pressure changes more than 0.3 hPa

This shows an important idea in embedded systems — converting complex data into simple visual signals that anyone can understand quickly.

In terms of **Electric Vehicles**, this project shows how simple systems can help in **battery safety, temperature control, and environmental monitoring**.

Through this project, important skills are learned, such as:

- Sensor interfacing
- I2C communication
- Display programming
- Logic-based LED control

These skills are useful in real-world **automotive and embedded systems engineering**.

With future improvements like **IoT connectivity, multiple sensors, smart thresholds, and CAN bus integration**, this project can be developed into a more advanced and practical system for EV applications.

— End of Report —

References

- Bosch Sensortec. BMP180 Digital Barometric Pressure Sensor — Data Sheet. Bosch GmbH, 2013.
- Adafruit Industries. Adafruit BMP085/BMP180 Library. github.com/adafruit/Adafruit-BMP085-Library
- Adafruit Industries. Adafruit SSD1306 OLED Library. github.com/adafruit/Adafruit_SSD1306
- Arduino. Arduino UNO R4 Minima — Official Documentation. docs.arduino.cc
- Rajkamal. Embedded Systems Architecture, Programming and Design. 3rd Ed. Tata McGraw-Hill, 2017.
- Andrea, D. Battery Management Systems for Large Lithium-Ion Battery Packs. Artech House, 2010.